```
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ nano m47.c
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ gcc m47.c -o m47
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ ./m47
==========================================
       MEMORY CLEANUP AND VALGRIND TEST
==========================================

[TEST 1] malloc()
malloc: Dynamic memory allocation test
malloc memory released successfully.

[TEST 2] calloc()
calloc values: 0 10 20 30 40 50 60 70 80 90
calloc memory released successfully.

[TEST 3] realloc()
realloc values: 1 2 3 4 5 6 7 8 9 10
realloc memory released successfully.

[TEST 4] String allocation
Source : Memory cleanup verification
Copy   : Memory cleanup verification
String memory released successfully.

==========================================
Memory cleanup tests completed.
All allocated resources were released.
==========================================
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$
```


```
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ nano m47.c
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ gcc m47.c -o m47
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ ./m47
==========================================
       MEMORY CLEANUP AND VALGRIND TEST
==========================================

[TEST 1] malloc()
malloc: Dynamic memory allocation test
malloc memory released successfully.

[TEST 2] calloc()
calloc values: 0 10 20 30 40 50 60 70 80 90
calloc memory released successfully.

[TEST 3] realloc()
realloc values: 1 2 3 4 5 6 7 8 9 10
realloc memory released successfully.

[TEST 4] String allocation
Source : Memory cleanup verification
Copy   : Memory cleanup verification
String memory released successfully.

==========================================
Memory cleanup tests completed.
All allocated resources were released.
==========================================
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$
```

```
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ cat m47.c
/* memory_cleanup_test.c
 *
 * Run Valgrind
 * Identify Memory Leaks
 * Fix Memory Errors
 * Release Resources
 * Verify Cleanup
 * Document Findings
 *
 * Compile:
 * gcc -g -Wall -Wextra memory_cleanup_test.c -o memory_cleanup_test
 *
 * Run normally:
 * ./memory_cleanup_test
 *
 * Run Valgrind:
 * valgrind --leak-check=full --show-leak-kinds=all ./memory_cleanup_test
 */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define SIZE 100

void test_malloc()
{
    char *buffer = malloc(SIZE);

    if (buffer == NULL)
    {
        perror("malloc");
        return;
    }

    strcpy(buffer, "Dynamic memory allocation test");

    printf("malloc: %s\n", buffer);

    free(buffer);
    buffer = NULL;

    printf("malloc memory released successfully.\n");
}

void test_calloc()
{
    int *numbers = calloc(10, sizeof(int));

    if (numbers == NULL)
    {
        perror("calloc");
        return;
    }

    for (int i = 0; i < 10; i++)
    {
        numbers[i] = i * 10;
    }

    printf("calloc values: ");
```

```
    printf("malloc memory released successfully.\n");
}

void test_calloc()
{
    int *numbers = calloc(10, sizeof(int));

    if (numbers == NULL)
    {
        perror("calloc");
        return;
    }

    for (int i = 0; i < 10; i++)
    {
        numbers[i] = i * 10;
    }

    printf("calloc values: ");

    for (int i = 0; i < 10; i++)
    {
        printf("%d ", numbers[i]);
    }

    printf("\n");

    free(numbers);
    numbers = NULL;

    printf("calloc memory released successfully.\n");
}

void test_realloc()
{
    int *numbers = malloc(5 * sizeof(int));

    if (numbers == NULL)
    {
        perror("malloc");
        return;
    }

    for (int i = 0; i < 5; i++)
    {
        numbers[i] = i + 1;
    }

    int *temp = realloc(numbers, 10 * sizeof(int));

    if (temp == NULL)
    {
        perror("realloc");
        free(numbers);
        return;
    }

    numbers = temp;

    for (int i = 5; i < 10; i++)
    {
        numbers[i] = i + 1;
    }

    printf("realloc values: ");
```

```
    free(numbers);
    numbers = NULL;

    printf("realloc memory released successfully.\n");
}

void test_string_memory()
{
    char *source = malloc(50);

    if (source == NULL)
    {
        perror("malloc");
        return;
    }

    strcpy(source, "Memory cleanup verification");

    char *copy = malloc(strlen(source) + 1);

    if (copy == NULL)
    {
        perror("malloc");
        free(source);
        return;
    }

    strcpy(copy, source);

    printf("Source : %s\n", source);
    printf("Copy   : %s\n", copy);

    free(source);
    source = NULL;

    free(copy);
    copy = NULL;

    printf("String memory released successfully.\n");
}

int main()
{
    printf("============================================\n");
    printf("       MEMORY CLEANUP AND VALGRIND TEST\n");
    printf("============================================\n");

    printf("\n[TEST 1] malloc()\n");
    test_malloc();

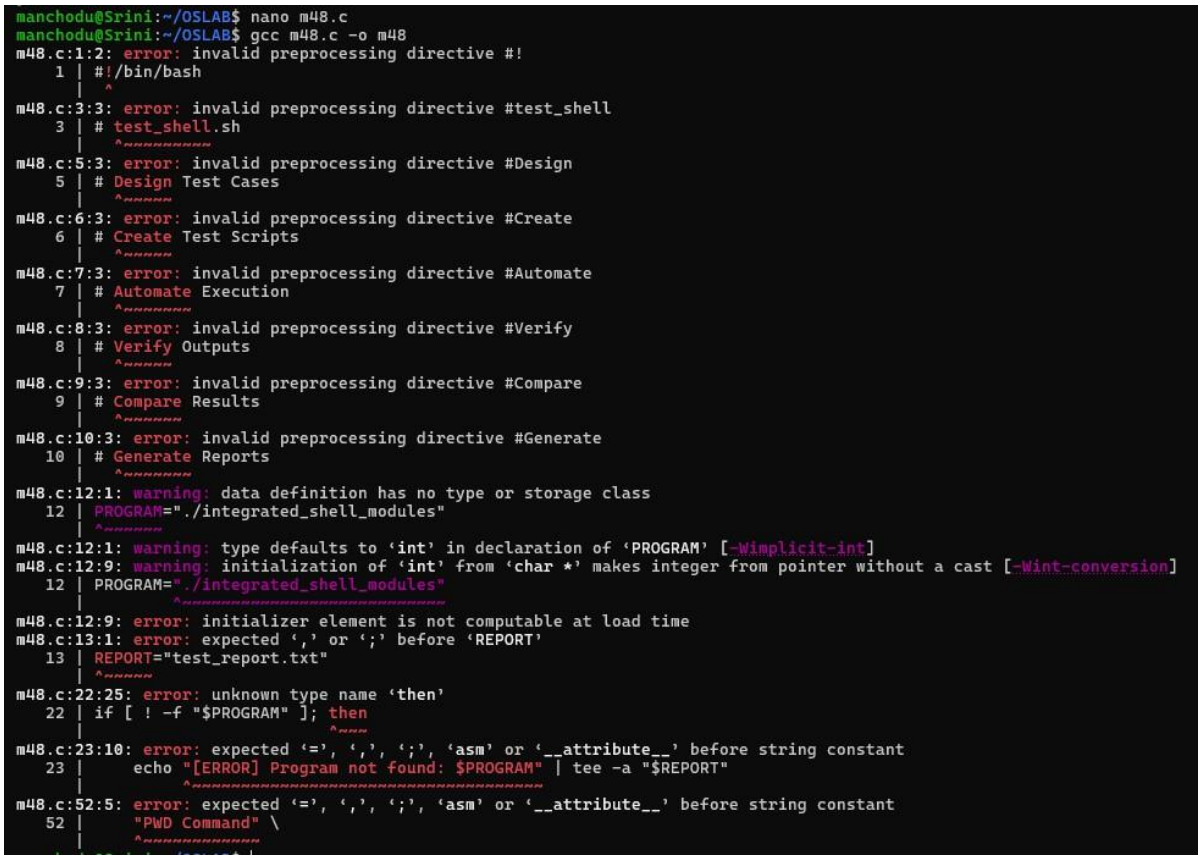
    printf("\n[TEST 2] calloc()\n");
    test_calloc();

    printf("\n[TEST 3] realloc()\n");
    test_realloc();

    printf("\n[TEST 4] String allocation\n");
    test_string_memory();

    printf("\n============================================\n");
    printf("Memory cleanup tests completed.\n");
    printf("All allocated resources were released.\n");
    printf("============================================\n");

    return 0;
}
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ |
```


manchodu@Srini:~/OSLAB$ nano m48.c
manchodu@Srini:~/OSLAB$ gcc m48.c -o m48
m48.c:1:2: error: invalid preprocessing directive #!
1 | #!/bin/bash
| ^
m48.c:3:3: error: invalid preprocessing directive #test_shell
3 | # test_shell.sh
| ^~~~~~~~~~~
m48.c:5:3: error: invalid preprocessing directive #Design
5 | # Design Test Cases
| ^~~~~~
m48.c:6:3: error: invalid preprocessing directive #Create
6 | # Create Test Scripts
| ^~~~~~
m48.c:7:3: error: invalid preprocessing directive #Automate
7 | # Automate Execution
| ^~~~~~~~
m48.c:8:3: error: invalid preprocessing directive #Verify
8 | # Verify Outputs
| ^~~~~~
m48.c:9:3: error: invalid preprocessing directive #Compare
9 | # Compare Results
| ^~~~~~~
m48.c:10:3: error: invalid preprocessing directive #Generate
10 | # Generate Reports
| ^~~~~~~~
m48.c:12:1: warning: data definition has no type or storage class
12 | PROGRAM="./integrated_shell_modules"
| ^~~~~~~
m48.c:12:1: warning: type defaults to ‘int’ in declaration of ‘PROGRAM’ [-Wimplicit-int]
m48.c:12:9: warning: initialization of ‘int’ from ‘char *’ makes integer from pointer without a cast [-Wint-conversion]
12 | PROGRAM="./integrated_shell_modules"
| ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
m48.c:12:9: error: initializer element is not computable at load time
m48.c:13:1: error: expected ‘,’ or ‘;’ before ‘REPORT’
13 | REPORT="test_report.txt"
| ^~~~~~
m48.c:22:25: error: unknown type name ‘then’
22 | if [! -f "$PROGRAM"]; then
| ^~~~
m48.c:23:10: error: expected ‘=’, ‘,’, ‘;’, ‘asm’ or ‘__attribute__’ before string constant
23 | echo "[ERROR] Program not found: $PROGRAM" | tee -a "$REPORT"
| ^~~~~~~
m48.c:52:5: error: expected ‘=’, ‘,’, ‘;’, ‘asm’ or ‘__attribute__’ before string constant
52 | "PWD Command" \
| ^~~~~~~~~~~~~~~~

```
#!/bin/bash

# test_shell.sh
#
# Design Test Cases
# Create Test Scripts
# Automate Execution
# Verify Outputs
# Compare Results
# Generate Reports

PROGRAM="./integrated_shell_modules"
REPORT="test_report.txt"

PASS=0
FAIL=0

echo "==========================================" | tee "$REPORT"
echo "       AUTOMATED SHELL TEST REPORT" | tee -a "$REPORT"
echo "==========================================" | tee -a "$REPORT"

if [ ! -f "$PROGRAM" ]; then
    echo "[ERROR] Program not found: $PROGRAM" | tee -a "$REPORT"
    echo "Compile the program first." | tee -a "$REPORT"
    exit 1
fi

run_test()
{
    TEST_NAME="$1"
    INPUT="$2"
    EXPECTED="$3"

    echo "" | tee -a "$REPORT"
    echo "TEST: $TEST_NAME" | tee -a "$REPORT"

    OUTPUT=$(printf "%b" "$INPUT" | timeout 10 "$PROGRAM" 2>&1)

    echo "$OUTPUT" >> "$REPORT"

    if echo "$OUTPUT" | grep -q "$EXPECTED"
    then
        echo "RESULT: PASS" | tee -a "$REPORT"
        PASS=$((PASS + 1))
    else
        echo "RESULT: FAIL" | tee -a "$REPORT"
        FAIL=$((FAIL + 1))
    fi
}

run_test \
    "PWD Command" \
    "pwd\nexit\n" \
    "shell"

run_test \
    "HELP Command" \
    "help\nexit\n" \
    "Available commands"

run_test \
    "Invalid Command" \
    "invalid_command_xyz\nexit\n" \
    "not found"

run_test \
    "Invalid Syntax" \
    "ls || grep txt\nexit\n" \
    "SYNTAX ERROR"

run_test \
```

```
    "Invalid Command" \
    "invalid_command_xyz\nexit\n" \
    "not found"

run_test \
    "Invalid Syntax" \
    "ls || grep txt\nexit\n" \
    "SYNTAX ERROR"

run_test \
    "Directory Change" \
    "cd /tmp\npwd\nexit\n" \
    "/tmp"

run_test \
    "Exit Command" \
    "exit\n" \
    "Shell exiting"

echo "" | tee -a "$REPORT"
echo "==========================================" | tee -a "$REPORT"
echo "              TEST SUMMARY" | tee -a "$REPORT"
echo "==========================================" | tee -a "$REPORT"

echo "Passed Tests : $PASS" | tee -a "$REPORT"
echo "Failed Tests : $FAIL" | tee -a "$REPORT"
echo "Total Tests  : $((PASS + FAIL))" | tee -a "$REPORT"

if [ "$FAIL" -eq 0 ]
then
    echo "Overall Result: PASS" | tee -a "$REPORT"
    exit 0
else
    echo "Overall Result: FAIL" | tee -a "$REPORT"
    exit 1
fi
```

```
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ nano m49.c
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ gcc m49.c -o m49
m49.c:1:2: error: invalid preprocessing directive #!
    1 | #!/bin/bash
      |  ^
m49.c:3:3: error: invalid preprocessing directive #run_all_tests
    3 | # run_all_tests.sh
      |   ^~~~~~~~~~~~~~
m49.c:5:3: error: invalid preprocessing directive #Complete
    5 | # Complete automated execution:
      |   ^~~~~~~~
m49.c:6:3: error: "1." after # is not a positive integer
    6 | # 1. Compile program
      |   ^~
m49.c:7:3: error: "2." after # is not a positive integer
    7 | # 2. Run functional tests
      |   ^~
m49.c:8:3: error: "3." after # is not a positive integer
    8 | # 3. Run Valgrind
      |   ^~
m49.c:9:3: error: "4." after # is not a positive integer
    9 | # 4. Generate final report
      |   ^~
m49.c:11:1: warning: data definition has no type or storage class
   11 | REPORT="final_test_report.txt"
      | ^~~~~~
m49.c:11:1: warning: type defaults to ‘int’ in declaration of ‘REPORT’ [-Wimplicit-int]
m49.c:11:8: warning: initialization of ‘int’ from ‘char *’ makes integer from pointer without a cast [-Wint-conversion]
   11 | REPORT="final_test_report.txt"
      |        ^~~~~~~~~~~~~~~~~~~~~~
m49.c:11:8: error: initializer element is not computable at load time
m49.c:13:1: error: expected ‘,’ or ‘;’ before ‘echo’
   13 | echo "==========================================" > "$REPORT"
      | ^~~~
```

```
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ cat m49.c
#!/bin/bash

# run_all_tests.sh
#
# Complete automated execution:
# 1. Compile program
# 2. Run functional tests
# 3. Run Valgrind
# 4. Generate final report

REPORT="final_test_report.txt"

echo "============================================" > "$REPORT"
echo "       COMPLETE TEST AUTOMATION REPORT" >> "$REPORT"
echo "============================================" >> "$REPORT"

echo "" >> "$REPORT"
echo "1. COMPILATION" >> "$REPORT"

gcc -g -Wall -Wextra memory_cleanup_test.c \
    -o memory_cleanup_test 2>> "$REPORT"

if [ $? -eq 0 ]
then
    echo "Compilation: PASS" >> "$REPORT"
else
    echo "Compilation: FAIL" >> "$REPORT"
    exit 1
fi

echo "" >> "$REPORT"
echo "2. FUNCTIONAL EXECUTION" >> "$REPORT"

./memory_cleanup_test >> "$REPORT" 2>&1

if [ $? -eq 0 ]
then
    echo "Functional Execution: PASS" >> "$REPORT"
else
    echo "Functional Execution: FAIL" >> "$REPORT"
fi

echo "" >> "$REPORT"
echo "3. VALGRIND MEMORY ANALYSIS" >> "$REPORT"

valgrind \
    --leak-check=full \
    --show-leak-kinds=all \
    ./memory_cleanup_test \
    >> "$REPORT" 2>&1

if grep -q "All heap blocks were freed -- no leaks are possible" "$REPORT"
then
    echo "" >> "$REPORT"
    echo "Memory Test: PASS - No memory leaks detected." >> "$REPORT"
else
    echo "" >> "$REPORT"
    echo "Memory Test: CHECK REQUIRED - Possible memory issues." >> "$REPORT"
fi

echo "" >> "$REPORT"
echo "============================================" >> "$REPORT"
echo "              TESTING COMPLETED" >> "$REPORT"
echo "============================================" >> "$REPORT"

cat "$REPORT"
```